Parallel and Distributed Computing

Assignment 6

Submitted to: Dr Saif Nalband
Submitted by: Anushka Gupta
Roll No:102303358
Course: UCS645

A. Write the device query code, compile and run it on your system. Query enough information to know all the details of your device. Example queries: GPU card's name, GPU computation capabilities, Maximum number of block dimensions, Maximum number of grid dimensions, Maximum size of GPU memory, Amount of constant and share memory, Warp size, etc. Answer the following questions in your report.

1. What is the architecture and compute capability of your GPU?

2. What are the maximum block dimensions for your GPU?

3. Suppose you are launching a one dimensional grid and block. If the hardware's maximum grid dimension is 65535 and the maximum block dimension is 512, what is the maximum number threads can be launched on the GPU?

4. Under what conditions might a programmer choose not want to launch the maximum number of threads?

5. What can limit a program from launching the maximum number of threads on a GPU?

6. What is shared memory? How much shared memory is on your GPU?

7. What is global memory? How much global memory is on your GPU?

8. What is constant memory? How much constant memory is on your GPU?

9. What does warp size signify on a GPU? What is your GPU's warp size?

10. Is double precision supported on your GPU?

Code:

```
%%writefile device_query.cu
#include <stdio.h>
#include <cuda_runtime.h>

int main() {
    cudaDeviceProp prop;
    int count;
    cudaGetDeviceCount(&count);
    printf("Number of CUDA devices: %d\n\n", count);

    for (int i = 0; i < count; i++) {
        cudaGetDeviceProperties(&prop, i);
        printf("=== Device %d: %s ===\n", i, prop.name);
        printf("Compute Capability: %d.%d\n", prop.major, prop.minor);
        printf("Max block dimensions: (%d, %d, %d)\n",
               prop.maxThreadsDim[0], prop.maxThreadsDim[1], prop.maxThreadsDim[2]);
        printf("Max grid dimensions: (%d, %d, %d)\n",
               prop.maxGridSize[0], prop.maxGridSize[1], prop.maxGridSize[2]);
        printf("Global memory: %.2f GB\n", prop.totalGlobalMem / (1024.0*1024*1024));
        printf("Constant memory: %lu bytes\n", prop.totalConstMem);
        printf("Shared memory per block: %lu bytes\n", prop.sharedMemPerBlock);
        printf("Warp size: %d\n", prop.warpSize);
        printf("Max threads per block: %d\n", prop.maxThreadsPerBlock);
        printf("Double precision supported: %s\n\n",
               (prop.major >= 2) ? "Yes" : "No");
    }
    return 0;
}
```

```
Overwriting device_query.cu
```

Results:

```
!nvcc -arch=sm_75 device_query.cu -o device_query
```

```
!./device_query
```

```
Number of CUDA devices: 1

=== Device 0: Tesla T4 ===
Compute Capability: 7.5
Max block dimensions: (1024, 1024, 64)
Max grid dimensions: (2147483647, 65535, 65535)
Global memory: 14.56 GB
Constant memory: 65536 bytes
Shared memory per block: 49152 bytes
Warp size: 32
Max threads per block: 1024
Double precision supported: Yes
```

The device query program was executed on Google Colab using NVIDIA Tesla T4 GPU to extract hardware details.

GPU Name: Tesla T4
Compute Capability: 7.5
Architecture: Turing
Global Memory: 14.56 GB
Shared Memory: 49152 bytes
Constant Memory: 65536 bytes
Warp Size: 32
Maximum Threads per Block: 1024
Maximum Block Dimensions: (1024, 1024, 64)

Answer 1:
The GPU used is Tesla T4, based on Turing architecture, with compute capability 7.5.

Answer 2:
The maximum block dimensions are (1024, 1024, 64).

Answer 3:
Maximum number of threads = 65535 × 512 = 33,553,920.

Answer 4:
A programmer may choose not to launch the maximum number of threads due to memory limitations, scheduling overhead, and resource contention, which can reduce performance.

Answer 5:
The number of threads can be limited by shared memory size, register usage, hardware constraints, and kernel configuration.

Answer 6:
Shared memory is a fast on-chip memory accessible by threads within the same block. The shared memory on this GPU is 49152 bytes.

Answer 7:
Global memory is a large memory accessible by all threads but is slower compared to shared memory. The global memory on this GPU is 14.56 GB.

Answer 8:
Constant memory is a read-only cached memory accessible by all threads. The constant memory on this GPU is 65536 bytes.

Answer 9:
Warp size signifies the number of threads executed together in a group. The warp size of this GPU is 32.

Answer 10:
Yes, double precision is supported on this GPU.

Conclusion:
The Tesla T4 GPU provides high parallel processing capability with large memory and efficient execution using warp-based architecture.

Observation:
The GPU supports a large number of parallel threads, which indicates high computational capability suitable for data-parallel tasks.

Inference:
The presence of a 32-thread warp size shows that execution follows SIMD-style parallelism, where threads execute in groups for efficiency. Shared memory being significantly smaller but faster than global memory suggests it should be used for performance-critical operations.

B) Write a CUDA program to calculate the sum of the elements in an array. The array contains single precision floating point numbers. Generate your input array. For this question, do the following.

1. Allocate device memory
2. Copy host memory to device
3. Initialize thread block and kernel grid dimensions
4. Invoke CUDA kernel

5. Copy results from device to host
6. Free device memory
7. Write the CUDA kernel that computes the sum.

Code:

```
%%writefile array_sum.cu
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

// 7. CUDA Kernel that computes the sum
__global__ void sumKernel(float *d_input, float *d_output, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        atomicAdd(d_output, d_input[idx]);   // Simple reduction using atomicAdd
    }
}

int main() {
    int N = 1 << 20;                    // 1,048,576 elements (you can change this)
    size_t size = N * sizeof(float);

    float *h_input = (float*)malloc(size);
    float *h_output = (float*)malloc(sizeof(float));

    // Generate input array (all 1.0f → expected sum = N)
    for (int i = 0; i < N; i++) {
        h_input[i] = 1.0f;
    }

    float *d_input, *d_output;

    // 1. Allocate device memory
    cudaMalloc((void**)&d_input, size);
    cudaMalloc((void**)&d_output, sizeof(float));

    // 2. Copy host memory to device
    cudaMemcpy(d_input, h_input, size, cudaMemcpyHostToDevice);
    cudaMemset(d_output, 0, sizeof(float));   // initialize output to 0
```

```
    // 3. Initialize thread block and kernel grid dimensions
    int threadsPerBlock = 256;
    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;

    // 4. Invoke CUDA kernel
    sumKernel<<<blocksPerGrid, threadsPerBlock>>>(d_input, d_output, N);

    cudaDeviceSynchronize();


    // 5. Copy results from device to host
    cudaMemcpy(h_output, d_output, sizeof(float), cudaMemcpyDeviceToHost);

    // 6. Free device memory
    cudaFree(d_input);
    cudaFree(d_output);

    printf("Sum of array = %.2f (Expected: %d)\n", *h_output, N);

    free(h_input);
    free(h_output);
    return 0;
}
```

```
Writing array_sum.cu
```

Results :

```
[17]
✓ 1s
!nvcc -arch=sm_75 array_sum.cu -o array_sum
!./array_sum

Sum of array = 1048576.00 (Expected: 1048576)
```

The CUDA program computes the sum of elements of a floating-point array using parallel processing on the GPU.

Step 1:
Device memory was allocated using cudaMalloc for both the input array and the output result.

Step 2:
The input array was initialised on the host with all elements as 1.0 and then copied to device memory using cudaMemcpy.

Step 3:
The thread block size was set to 256, and the grid size was calculated based on the total number of elements to ensure all elements are processed.

Step 4:
The CUDA kernel was launched, where each thread computes its global index and processes one element of the array.

Step 5:
atomicAdd was used inside the kernel to safely add each element to a single result variable, avoiding race conditions.

Step 6:
cudaDeviceSynchronize was used to ensure all threads completed execution before copying the result.

Step 7:
The final result was copied back from device to host using cudaMemcpy.

Step 8:
Allocated device memory was freed using cudaFree.

Kernel Explanation:
Each thread calculates its index using block and thread IDs and adds the corresponding array element to a global result using atomicAdd. This ensures correct accumulation in a parallel environment.

Output:
Sum of array = 1048576.00 (Expected: 1048576)

Inference:
The program demonstrates parallel execution where multiple threads process data simultaneously, significantly improving computation speed compared to sequential execution.

Limitation:
The use of atomicAdd introduces serialisation, which can reduce performance for very large datasets. More optimised reduction techniques can be used for better efficiency.

Conclusion:
CUDA enables efficient parallel computation on a GPU, and the program successfully computes the sum of a large array using parallel threads.

C) Perform Matrix Addition of two large integer matrices in CUDA. Answer the following questions.

1. How many floating operations are being performed in the matrix addition kernel?

2. How many global memory reads are being performed by your kernel?

3. How many global memory writes are being performed by your kernel?

Code :

[18] ✓ 0s

```
%%writefile matrix_add.cu
#include <stdio.h>
#include <cuda_runtime.h>

#define N 1024   // Large matrix size (1024 x 1024)

__global__ void matrixAddKernel(int *A, int *B, int *C, int width) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < width && col < width) {
        int idx = row * width + col;
        C[idx] = A[idx] + B[idx];     // one integer addition per element
    }
}

int main() {
    size_t size = N * N * sizeof(int);

    int *h_A = (int*)malloc(size);
    int *h_B = (int*)malloc(size);
    int *h_C = (int*)malloc(size);

    // Initialize matrices
    for (int i = 0; i < N * N; i++) {
        h_A[i] = i % 100;
        h_B[i] = i % 50;
    }

    int *d_A, *d_B, *d_C;

    // Allocate device memory
    cudaMalloc((void**)&d_A, size);
    cudaMalloc((void**)&d_B, size);
    cudaMalloc((void**)&d_C, size);
```

```
    // Copy host to device
    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

    // Thread and grid configuration
    dim3 threadsPerBlock(16, 16);
    dim3 blocksPerGrid((N + 15)/16, (N + 15)/16);

    // Launch kernel
    matrixAddKernel<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);

    // Copy result back
    cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

    // Free device memory
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    printf("Matrix addition completed successfully!\n");
    printf("Sample result C[0][0] = %d\n", h_C[0]);

    free(h_A);
    free(h_B);
    free(h_C);
    return 0;
}
```

```
Writing matrix_add.cu
```

Result:

```
Writing matrix_add.cu
```

```
!nvcc -arch=sm_75 matrix_add.cu -o matrix_add
!./matrix_add
```

```
Matrix addition completed successfully!
Sample result C[0][0] = 0
```

The CUDA program performs matrix addition of two large integer matrices using parallel processing on the GPU.

Each thread computes one element of the result matrix by adding corresponding elements from the two input matrices.

Thread Configuration:
A 2D grid and 2D block structure was used with block size 16 × 16 to efficiently map threads to matrix elements.

Working:
Each thread calculates its row and column index using block and thread indices. The corresponding linear index is computed and used to perform addition.

Answer 1:
There are 0 floating-point operations as the kernel performs integer addition.
Each thread performs one integer addition.
Total integer operations = N × N = 1024 × 1024 = 1,048,576 additions.

Answer 2:
Each thread reads two elements from global memory (one from each input matrix).
Total global memory reads = 2 × N × N = 2 × 1024 × 1024 = 2,097,152 reads.

Answer 3:
Each thread writes one result to global memory.
Total global memory writes = N × N = 1024 × 1024 = 1,048,576 writes.

Output:
Matrix addition completed successfully.

Inference:
The program demonstrates efficient parallel computation where each thread handles one element independently, leading to high parallelism.

Conclusion:
Matrix addition is highly parallelizable, and CUDA efficiently distributes the computation across threads, resulting in faster execution compared to sequential processing.